

Conteúdo

Trabalho Prático — Jogo “Tâb”	2
1. Regras do jogo (Tâb)	2
2. Primeira entrega — cliente (single-page app, só contra o computador)	3
3. Segunda entrega — protocolo cliente/servidor (jogo distribuído)	4
4. Envio para avaliação (mecanismo de submissão)	7
5. Grupos e apresentação	7
6. FAQ (perguntas frequentes, já respondidas pelo docente)	8
O que ainda falta / está por confirmar	8

Trabalho Prático — Jogo “Tâb”

O trabalho prático da disciplina consiste em implementar, em grupos de 3 alunos, uma versão web do jogo de tabuleiro tradicional **Tâb**, em 2 entregas avaliadas (que correspondem a 3 fases de implementação — ver nota sobre a “2ª entrega” mais abaixo). Ver `CALENDARIO.md` para os prazos.

1. Regras do jogo (Tâb)

(Há variantes deste jogo — para o trabalho contam só estas regras.)

Tabuleiro e peças

- Tabuleiro com **4 linhas**; o número de colunas é sempre **ímpar**, entre 7 e 15 (frequentemente 9).
- Cada jogador tem tantas peças quantas colunas o tabuleiro tiver (**size** peças cada); as peças de cada jogador têm cor diferente do adversário e começam na linha do seu próprio lado (1ª linha para um jogador, 4ª para o outro).
- **Perspetiva**: cada jogador vê o tabuleiro com as suas peças em baixo — o tabuleiro do adversário está rodado 180º em relação ao seu.
- **Estado de cada peça** (relevante para as regras de movimento):
 1. ainda não foi movida;
 2. já foi movida mas não chegou à última linha (linha do adversário);
 3. já esteve na última linha.

Dado de paus

Em vez de um dado tradicional, usam-se **4 paus** com duas faces distintas (resultam de um pau maior cortado a meio ao comprimento, e cada metade cortada a meio longitudinalmente): uma face plana e clara (interior do pau), outra escura e arredondada (casca). Apertam-se os 4 na mão e, abrindo-a, cada um cai numa das duas faces. Conta-se quantos ficam com a face clara para cima.

Nº de faces claras	Valor do dado	Nome	Repete jogada?	Probabilidade aprox.
0	6	Sitteh	sim	6%
1	1	Tâb	sim	25%
2	2	Itneyn	não	38%
3	3	Teláteh	não	25%
4	4	Arba'ah	sim	6%

Repara que **não existe o valor 5** — a soma de faces claras (0 a 4) só coincide com o “valor” do dado exceto no caso 0, que vale 6. Os valores 1, 4 e 6 dão direito a jogar (mover) e depois **repetir** o lançamento.

Mover as peças

- **Início do movimento**: a primeira vez que uma peça é movida, o dado tem de dar exatamente **1 (Tâb)** — se sair 4 ou 6 (que dão direito a repetir), repete-se o lançamento; a casa de destino não pode já ter uma peça do próprio jogador. Como consequência, começa-se sempre a mover as peças mais à direita.
- **Progressão pelo tabuleiro** (sentido de “cobra”):
 - 1ª e 3ª linhas: esquerda → direita;
 - 2ª e 4ª linhas: direita → esquerda;
 - ao sair da 1ª linha, passa-se para a 2ª; ao sair da 2ª, para a 3ª;
 - ao sair da 3ª linha, pode passar-se para a **4ª OU** voltar à **2ª** (única bifurcação do percurso);
 - ao sair da 4ª linha, volta-se à 3ª.
- **Restrições**:

- só pode haver **uma peça por casa**;
- uma peça só entra na 4ª linha **uma única vez**;
- uma peça na 4ª linha só se pode voltar a mover se o jogador **não** tiver mais nenhuma peça na sua própria linha inicial (1ª linha);
- uma peça **nunca** pode regressar à linha inicial do seu jogador.
- **Passar a vez:** só é permitido passar se não houver jogada válida possível e não for possível lançar o dado novamente (isto é, o jogador fica mesmo sem opções).

Capturar peças

Se o destino de uma jogada tiver uma peça do adversário, essa peça é **capturada** e desaparece do tabuleiro.

Fim do jogo

O jogo termina quando um jogador fica sem peças; ganha o outro.

2. Primeira entrega — cliente (single-page app, só contra o computador)

Prazo: ver CALENDARIO.md (30 out / tolerância 2 nov). **Vale 3 valores.**

Requisito estrutural: a aplicação é uma **single-page application** — uma única página HTML, sem carregar outras páginas. CSS num ou mais ficheiros próprios; JavaScript num ficheiro próprio. HTML e CSS **têm de validar** (erros de validação são penalizados).

A página deve ter estas áreas (não têm de estar todas visíveis ao mesmo tempo — podem sobrepor-se temporariamente):

Área	O que faz
Logotipo	Nome do jogo em destaque (texto formatado ou imagem)
Configuração	Tamanho do tabuleiro; jogo vs. computador (só isto nesta entrega — vs. outro jogador é só na 2ª); primeiro a jogar; nível de IA
Comandos	Ver instruções; iniciar jogo; passar a vez (só se não houver jogada); desistir; ver classificações
Identificação	Área já criada/formatada para login (id + senha) — mas sem autenticação real ainda nesta entrega
Dado	Área clicável para lançar o dado de paus e mostrar o resultado (idealmente as faces de cada pau); reverte ao estado inicial no fim de cada jogada
Tabuleiro	A área mais complexa — ver abaixo
Instruções	Mostra as regras do jogo; pode ser um painel sobreposto, com comando para abrir/fechar
Classificações	Tabela dos melhores resultados (nesta entrega, só jogos vs. computador); painel sobreposto tal como instruções
Mensagens	Feedback textual: “é a tua vez”, “fizeste uma captura”, “jogada inválida”, “fim de jogo”, “desististe”, etc.

Tabuleiro — como implementar: 1. Fase 1: codificar em HTML puro (contentores para linhas/peças), formatado em CSS. 2. Fase 2: gerar o tabuleiro **a partir de JavaScript** usando a DOM e a sintaxe JS do CSS — assim dá para gerar tabuleiros de tamanhos diferentes consoante a configuração. **É valorizada uma abordagem orientada a objetos** para representar o tabuleiro/jogo. 3. **Modos de interação:** clicar numa casa tem efeitos diferentes consoante o momento — normalmente uma peça joga-se de imediato ao clicar;

mas se a peça estiver na 3ª linha (pode ir para 4ª ou 2ª), o clique seguinte escolhe o destino. Usar variáveis de estado para controlar isto, e mostrar sempre ao utilizador em que modo está (cursor e/ou mensagem).

IA (obrigatória nesta entrega): jogadas calculadas localmente no cliente. Abordagem simples: escolher aleatoriamente entre as jogadas válidas. Mais sofisticada: preferir jogadas que capturem. Podes combinar ambas (escolhida aleatoriamente a cada jogada) para simular níveis de dificuldade diferentes.

(Existe também um guia prático passo-a-passo desta entrega em `conteudo_site/praticas/folhas/entrega1/` — inclui um exemplo de modelo de dados em JS para o tabuleiro, se quiseres uma referência mais concreta.)

3. Segunda entrega — protocolo cliente/servidor (jogo distribuído)

Prazo: ver CALENDARIO.md (11 dez / tolerância 14 dez). **Vale 5 valores no total** — cobre as duas partes abaixo (protocolo + servidor Node), que no site aparecem como “entrega2” e “entrega3” mas são entregas e avaliadas em conjunto nesta única data.

Objetivo: tornar o jogo **distribuído** — dois jogadores em computadores diferentes, comunicando através de um servidor.

3.1 Protocolo (o que o teu cliente deve falar com o servidor)

Serviço web em `http://twserver.alunos.dcc.fc.up.pt:8008/`. Todos os pedidos são `fetch/XMLHttpRequest` com **POST** e dados em **JSON**, exceto `update`, que usa **GET** com dados `urlencoded` via **Server-Sent Events** (é a forma como o servidor notifica os clientes de eventos do jogo, sem eles terem de andar a perguntar).

Tabela de pedidos (= argumento necessário):

Função	group	nick	password	size	game	cell	Descrição
<code>register</code>							Regista utilizador com senha
<code>join</code>							Junta jogadores para iniciar jogo
<code>leave</code>							Desistir de jogo não terminado
<code>roll</code>							Lança o dado de paus
<code>pass</code>							Passa a vez
<code>notify</code>							Notifica o servidor de uma jogada
<code>update †</code>							GET/SSE — atualizações do jogo
<code>ranking</code>							Tabela classificativa

Significado de cada argumento: - `group` — nº do vosso grupo (ver pauta da 1ª entrega); serve só para isolar os testes do vosso grupo dos de outros grupos no servidor partilhado (há tabelas de ranking separadas por grupo). - `nick` / `password` — identificação e autenticação do jogador. - `size` — nº de colunas do tabuleiro (o tabuleiro tem `4 * size` casas). - `game` — hash identificador do jogo, devolvido por `join`. - `cell` (chamado `move` nalguns sítios) — casa do tabuleiro envolvida na jogada: quando `step == "from"`, é a casa de origem; quando `step == "to"`, é a casa de destino (escolher a mesma casa de novo **cancela** a seleção e volta ao passo `from`).

Comportamento importante de cada função: - `join`: empareceita 2 jogadores à espera do mesmo tamanho de tabuleiro; se ninguém estiver à espera, o jogador fica registado a aguardar. A função **retorna logo** (só dá o `game id`) — o emparelhamento em si é notificado a ambos via `update`. - `leave`: se ainda estiver à espera de adversário, sem consequências; se já em jogo, dá a vitória ao adversário. **Timeout automático de 2 minutos:** se não jogares a tempo, é como se tivesses feito `leave`. - `roll` / `pass` / `notify`: todos retornam de imediato (corpo `{}` se tudo correr bem); o resultado real da jogada chega depois a **ambos** os jogadores via `update`. - `register`: também serve para verificar login (reenviar `nick+password` já registados = sucesso; `password` errada = erro).

Todas as funções podem devolver `{"error": "mensagem"}` com status HTTP diferente de 200 (ver secção 4 abaixo, códigos concretos).

Tabela de respostas (campos que aparecem no JSON de resposta/evento, indica em que função aparece — a maioria só faz sentido em **update**):

Campo	register	join	leave	notify	update	ranking	O que é
cell							objeto {square, position} da última casa movida
dice							null, ou {stickValues[], value, keepPlaying}
error							mensagem de erro
game							id do jogo criado/em- parelhado
initial							nick do jogador cujas peças começam nas posições 0..size-1
mustPass							true/false — o jogador tem de passar
pieces							array com 4*size posições (peça ou null)
players							{nick1: "Blue"/"Red", nick2: ...}
ranking							lista [{nick, games, victories}, ...], máx. 10, ordenada
selected							array de índices de casas destacáveis (última jogada)

Campo	register	join	leave	notify	update	ranking	O que é
step							"from" / "to" / "take" — próxima escolha
turn							esperada nick de quem tem a vez
winner							nick do vencedor, ou null

Detalhe de peças (o array do estado do tabuleiro): $4 \times \text{size}$ posições; posição 0 = canto inferior direito na perspectiva do jogador initial. Cada posição é null (vazia) ou {color: "Blue"|"Red", inMotion: bool, reachedLastRow: bool}.

Exemplo de sessão completa (registro → emparelhamento → jogo → vitória), simplificado a partir dos exemplos do enunciado:

```
POST .../register {"nick":"zp","password":"secret"}          → {}
POST .../register {"nick":"jpleal","password":"another"}    → {}

POST .../join {"group":99,"nick":"zp","password":"secret","size":9}
                                                    → {"game":"fa93b40..."}
GET  .../update?nick=zp&game=fa93b40...                (SSE, fica à espera)

POST .../join {"group":99,"nick":"jpleal","password":"another","size":9}
                                                    → {"game":"fa93b40..."}
      (emparelhado com o jogo de zp que estava à espera)

→ evento update (para ambos): {"pieces":[...], "initial":"zp",
    "step":"from", "turn":"zp", "players":{"zp":"Blue","jpleal":"Red"}}

POST .../roll {"nick":"zp","password":"secret","game":"fa93b40..."} → {}
→ evento update: {"dice":{"stickValues":[false,true,false,false],
    "value":1,"keepPlaying":true}, "turn":"zp", "mustPass":null}
    (saiu Táb=1, zp pode jogar a peça mais à direita)

POST .../notify {"nick":"zp","password":"secret","game":"fa93b40...","cell":8}
                                                    → {}
→ evento update: {"cell":8,"selected":[8,9],"step":"from",
    "turn":"jpleal", ...}    (jogada feita, passa a vez)

... (o jogo continua até um jogador ficar sem peças) ...

→ evento update final: {"winner":"jpleal", "pieces":[...]}
    (não esquecer de fechar o EventSource de cada jogo no cliente)
```

Há mais exemplos completos (jogadas inválidas, escolher destino na 3ª linha, reverter seleção, tabela de ranking) em `conteudo_site/praticas/trabalho/entrega2/exemplos/`.

3.2 Servidor Node.js (substitua o servidor de testes pelo vosso)

- Ficheiro principal chamado **index.js**, estruturado em módulos.
- Deve implementar exatamente as mesmas funções/respostas descritas acima (3.1), validando tipos e obrigatoriedade de todos os argumentos.
- **Códigos de estado HTTP a devolver:**
 - 200 — pedido bem sucedido
 - 400 — erro no pedido (ex.: falha de validação)
 - 401 — não autorizado (**nick/password** inválidos)
 - 404 — pedido desconhecido
- **Persistência:** dados serializados em JSON, guardados em ficheiro via módulo **fs** (não é preciso base de dados a sério).
- **Hash do game:** usar o módulo **crypto**, MD5 em hexadecimal:

```
const crypto = require('crypto');
const hash = crypto.createHash('md5').update(value).digest('hex');
```

As passwords também devem ser cifradas antes de guardadas (mesmo módulo **crypto**).

- **Objetivo mínimo:** implementar **register** e **ranking**. As restantes (**join**, **leave**, **notify**, **update**) só devem ser tentadas depois, e contam como valorização para nota mais alta.
- **Publicação obrigatória** em **twserver.alunos.dcc.fc.up.pt**, porta 81XX (XX = nº do grupo):
 - Acesso SSH (2 autenticações, jump server + back-end):
`ssh -J up<nº>@ssh.alunos.dcc.fc.up.pt up<nº>@twserver-be`
 - Copiar ficheiros com **scp** (mesmo esquema de jump):
`scp -J up<nº>@ssh.alunos.dcc.fc.up.pt * up<nº>@twserver-be:pasta`
 - Autenticação: credenciais LDAP do LabCC.

Submissão desta entrega: ZIP com os módulos JS do servidor + **index.js** na raiz, **mais** **index.html/CSS/JS** do cliente (da 1ª entrega) já configurados para falar com o **vosso** servidor Node publicado. A nota da 1ª entrega não é reavaliada aqui.

4. Envio para avaliação (mecanismo de submissão)

- Submissão **online**, ficheiro **ZIP**, por qualquer elemento do grupo; podes submeter várias vezes (só conta a última).
- O ZIP deve conter, **sem pasta de topo**:
 - **index.html** na raiz;
 - um ou mais **.css**;
 - um ou mais **.js**.
- Criar com:

```
cd $PASTA_TRABALHO
zip trabalho.zip index.html *.{css,js}
```
- Submeter em: <https://mooshak.dcc.fc.up.pt/~tw/cgi-bin/execute?login> (login = credenciais LDAP do LabCC). Se aceite, aparece “Accepted”; se der “Compile Time Error”, clicar na mensagem para ver a causa.

5. Grupos e apresentação

- **Lista de grupos:** Google Sheet partilhada (usar Slack para encontrar colegas sem grupo).
- **Apresentação:** na semana seguinte ao prazo de cada entrega, tens de te inscrever num calendário partilhado escolhendo o número do grupo e o modo (presencial ou remoto) num horário livre.

6. FAQ (perguntas frequentes, já respondidas pelo docente)

- **Não arranjo grupo:** tenta primeiro outra pessoa sem grupo, mesmo de outra turma; em último caso, podes fazer sozinho (grupos podem ir até 3 este ano, dado o nº de inscritos).
- **Tamanhos de tabuleiro:** não são fixos — deve haver um seletor com várias opções (ex.: 2, 3, 4, 5 colunas... na prática lembra que size tem de ser ímpar entre 7–15, ver regras). Tabuleiro gerado em JS via DOM; formatação/posicionamento em CSS.
- **Frameworks (JS ou CSS): não podem ser usadas** — o trabalho tem de ser feito “à mão”.
- **CSS com features que o validador rejeita:** evitar — comportamento inconsistente entre navegadores, e além disso **erros de CSS são penalizados** na avaliação.
- **Conteúdo da tabela de classificação:** livre, desde que faça sentido com o jogo (ex.: vitórias/derrotas por tamanho de tabuleiro).
- **A tabela de classificação perde-se ao reiniciar o navegador:** normal na 1ª entrega (é focada em formatação); na 2ª entrega podes usar mecanismos de persistência do lado do cliente para jogos locais, e para jogos com servidor o ranking vem do servidor.

O que ainda falta / está por confirmar

- O nó “Valorizações” da 2ª entrega (entrega2/valorizacoes) está **sem conteúdo publicado** no site (dá página 404) — pode aparecer mais tarde.
- Não encontrei ainda o teu nº de grupo nem a tua turma — assim que souberes, digo-te para atualizar isto (o `group` é usado como argumento em vários pedidos do protocolo).